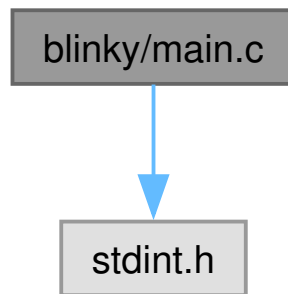

```

00001 /*
00002 * blinky/linker.ld -- Minimal linker script for RX72N bare-metal blinky.
00003 *
00004 * RX72N memory map (R5F572NNHxFB, 144-pin):
00005 *   Internal RAM : 0x00000000 - 0x0007FFFF (512 KB)
00006 *   Internal ROM : 0xFFE00000 - 0xFFFFFFFF (2 MB active region)
00007 *
00008 * Note: RAM starts at 0x4 not 0x0. The first 4 bytes (0x0-0x3) are reserved
00009 * by the hardware (used as the undefined instruction trap vector on RX).
00010 *
00011 * Fixed hardware vector locations (RX CPU specification):
00012 *   0xFFFFFFF80 - 0xFFFFFFFFC : Exception/interrupt vector table
00013 *   0xFFFFFFFFC - 0xFFFFFFFF : Reset vector (4 bytes, points to startup code)
00014 */
00015
00016 MEMORY
00017 {
00018     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC /* 512 KB - 4 bytes */
00019     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000 /* 2 MB */
00020 }
00021
00022 SECTIONS
00023 {
00024     /*
00025     * Fixed reset vector at the very top of flash.
00026     * The CPU reads this 32-bit address on power-on and jumps there.
00027     */
00028     .fvecvectors 0xFFFFFFF80 : AT(0xFFFFFFF80)
00029     {
00030         KEEP(*(.fvecvectors))
00031     } > ROM
00032
00033     /* Code and read-only data starting at base of ROM */
00034     .text 0xFFE00000 : AT(0xFFE00000)
00035     {
00036         *(.text*)
00037         *(.rodata*)
00038         etext = .;
00039     } > ROM
00040
00041     /* Uninitialised data (BSS) -- main.c has no globals, so this is empty */
00042     .bss :
00043     {
00044         _bss_start = .;
00045         *(.bss*)
00046         *(COMMON)
00047         _bss_end = .;
00048     } > RAM
00049
00050     /*
00051     * Stack definitions.
00052     * USP (user stack): top of RAM, grows downward.
00053     * ISP (interrupt stack): 2 KB below USP, grows downward.
00054     */
00055     __ustack = ORIGIN(RAM) + LENGTH(RAM);
00056     __istack = __ustack - 0x800;
00057 }

```

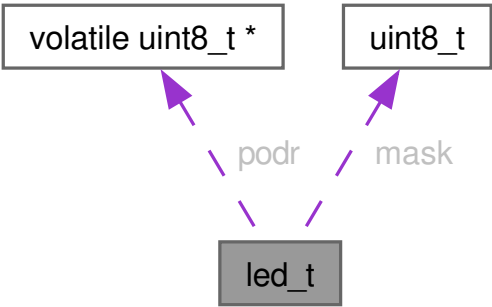
```
#include <stdint.h>
```



```
<<  
<<<<  
<<<<<<
```

```
*  
*  
*  
*  
*  
*  
  
    count  
*  
*  
*  
    led  
*  
    led  
*  
    k_flash_count
```

>>>>>>>



*		

```
enum breathe_params_t : uint32_t
```

--	--

```
00076      : uint32_t {
00077   k_max_bright  = 50U, /**< PWM duty-cycle upper bound (steps 0..50) */
00078   k_reps_per_step = 5U, /**< PWM cycles held at each brightness step */
00079   k_flash_count   = 3U, /**< Completion flashes after one full cycle */
00080   k_flash_on_cnt  = 8000U, /**< Flash ON busy-wait count */
00081   k_flash_off_cnt = 8000U, /**< Flash OFF busy-wait count */
00082 } breathe_params_t;
```

```
enum led_count_t : uint8_t
```

--	--

```
00087      : uint8_t {
00088   k_num_leds = 6U, /**< LEDs in the breathing sequence */
00089 } led_count_t;
```

```
enum port7_pins_t : uint8_t
```


```
00055      : uint8_t {
00056   k_pin_p71 = (uint8_t)(1U « 1U), /**< PORT7 pin 1 */
00057   k_pin_p72 = (uint8_t)(1U « 2U), /**< PORT7 pin 2 */
00058 } port7_pins_t;
```

```
enum port_addr_t : uintptr_t
```

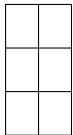

```
00039      : uintptr_t {
00040   k_port7_pdr_addr = 0x0008C007U, /**< PORT7 direction register */
00041   k_port7_podr_addr = 0x0008C027U, /**< PORT7 output data register */
00042   k_porta_pdr_addr = 0x0008C00AU, /**< PORTA direction register */
00043   k_porta_podr_addr = 0x0008C02AU, /**< PORTA output data register */
00044   k_portb_pdr_addr = 0x0008C00BU, /**< PORTB direction register */
00045   k_portb_podr_addr = 0x0008C02BU, /**< PORTB output data register */
00046 } port_addr_t;
```

```
enum porta_pins_t : uint8_t
```



```
00051         : uint8_t {
00052     k_pin_pa7 = (uint8_t)(1U « 7U), /**< PORTA pin 7 */
00053 } porta_pins_t;
```

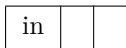
```
enum portb_pins_t : uint8_t
```



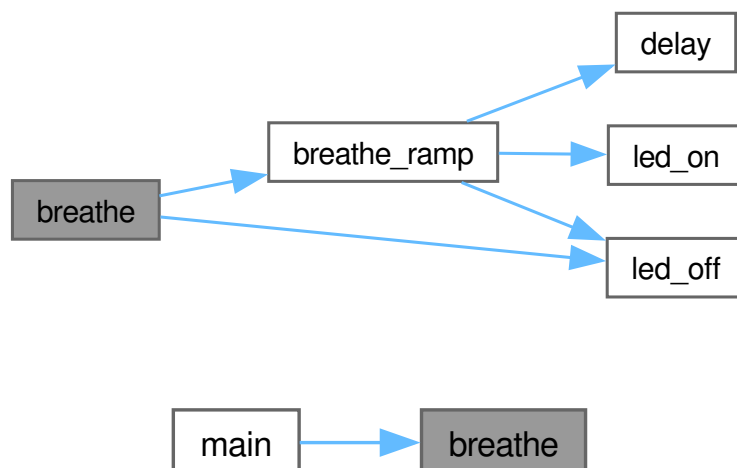
```
00060         : uint8_t {
00061     k_pin_pb0 = (uint8_t)(1U « 0U), /**< PORTB pin 0 */
00062     k_pin_pb1 = (uint8_t)(1U « 1U), /**< PORTB pin 1 */
00063     k_pin_pb2 = (uint8_t)(1U « 2U), /**< PORTB pin 2 */
00064 } portb_pins_t;
```

```
void breathe (
    const led_t * led)  [static]
```

```
led
```



```
00199 {
00200     breathe_ramp(led, 0U); /** in */
00201     breathe_ramp(led, 1U); /** out */
00202     led_off(led);
00203 }
```



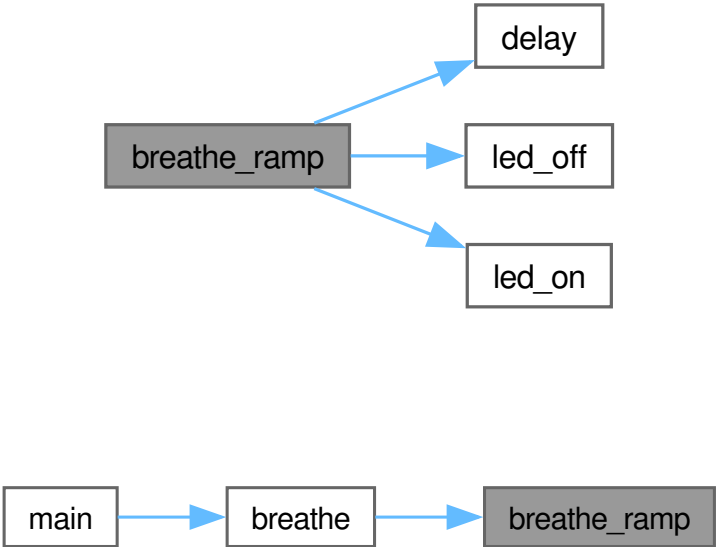
```
void breathe_ramp (
    const led_t * led,
    uint32_t invert) [static]

led

on_time k_max_bright invert k_reps_per_step
```

in		
in		>>

```
00172 {
00173     uint32_t b;
00174     uint32_t rep;
00175     uint32_t on_time;
00176     uint32_t off_time;
00177
00178     for (b = 0U; b <= k_max_bright; b++) {
00179         on_time = (invert == 0U) ? b : (k_max_bright - b);
00180         off_time = k_max_bright - on_time;
00181         for (rep = 0U; rep < k_reps_per_step; rep++) {
00182             if (on_time > 0U) {
00183                 led_on(led);
00184                 delay(on_time);
00185             }
00186             led_off(led);
00187             if (off_time > 0U) {
00188                 delay(off_time);
00189             }
00190         }
00191     }
00192 }
```

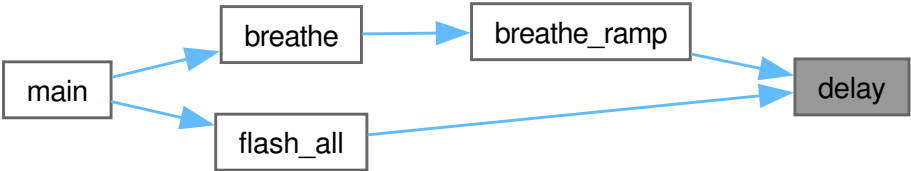



```
void delay (
    uint32_t count) [static]

count
```

in		~
----	--	---

```
00138 {
00139     volatile uint32_t i;
00140     for (i = 0U; i < count; i++) {
00141         __asm__ __volatile__("nop");
00142     }
00143 }
```

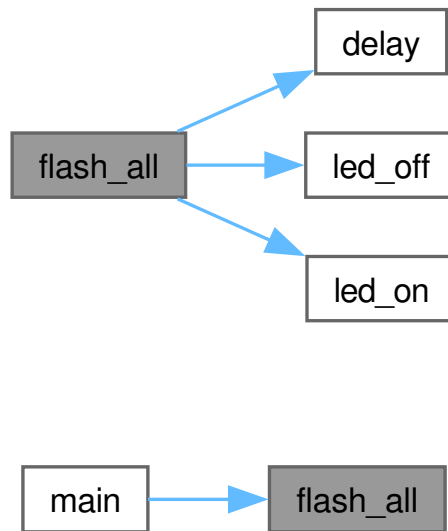


```
void flash_all (
    const led_t * leds,
    uint8_t count) [static]

k_flash_count
```

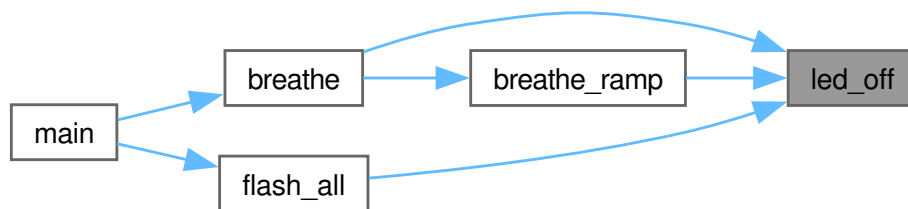
in		
in		leds

```
00214 {
00215     uint32_t f;
00216     uint8_t i;
00217
00218     for (f = 0U; f < k_flash_count; f++) {
00219         for (i = 0U; i < count; i++) {
00220             led_on(&leds[i]);
00221         }
00222         delay(k_flash_on_cnt);
00223
00224         for (i = 0U; i < count; i++) {
00225             led_off(&leds[i]);
00226         }
00227         delay(k_flash_off_cnt);
00228     }
00229 }
```



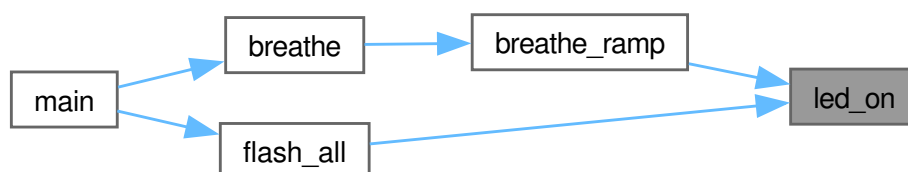
```
void led_off (
    const led_t * led)  [inline], [static]
```

```
00154 {
00155     *led->podr &= (uint8_t)~led->mask;
00156 }
```



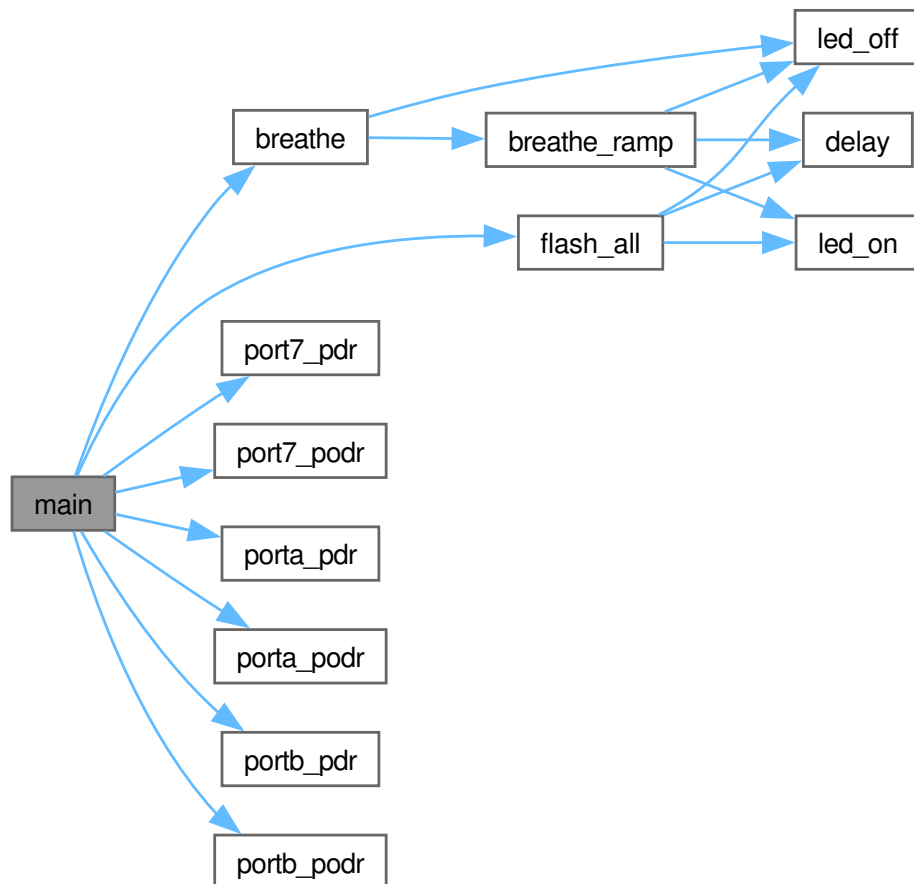
```
void led_on (
    const led_t * led)  [inline], [static]
```

```
00149 {
00150     *led->podr |= led->mask;
00151 }
```



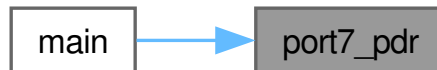
```
int main (
    void )
```

```
00239 {
00240     /* Set LED pins as outputs (PDR bit = 1). */
00241     *porta_pdr() |= (uint8_t)k_pin_pa7;
00242     *port7_pdr() |= (uint8_t)(k_pin_p71 | k_pin_p72);
00243     *portb_pdr() |= (uint8_t)(k_pin_pb0 | k_pin_pb1 | k_pin_pb2);
00244
00245     /* LED table -- order defines the breathing sequence. */
00246     const led_t leds[k_num_leds] = {
00247         { porta_podr(), (uint8_t)k_pin_pa7 },
00248         { portb_podr(), (uint8_t)k_pin_pb0 },
00249         { port7_podr(), (uint8_t)k_pin_p71 },
00250         { port7_podr(), (uint8_t)k_pin_p72 },
00251         { portb_podr(), (uint8_t)k_pin_pb1 },
00252         { portb_podr(), (uint8_t)k_pin_pb2 },
00253     };
00254
00255     for (;;) {
00256         uint8_t i;
00257         for (i = 0U; i < k_num_leds; i++) {
00258             breathe(&leds[i]);
00259         }
00260         flash_all(leds, k_num_leds);
00261     }
00262 }
```



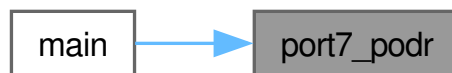
```
volatile uint8_t * port7_pdr (  
    void )    [inline], [static]
```

```
00106 {  
00107     return (volatile uint8_t *)k_port7_pdr_addr;  
00108 }
```



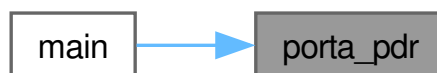
```
volatile uint8_t * port7_podr (  
    void )    [inline], [static]
```

```
00110 {  
00111     return (volatile uint8_t *)k_port7_podr_addr;  
00112 }
```



```
volatile uint8_t * porta_pdr (  
    void )    [inline], [static]
```

```
00114 {  
00115     return (volatile uint8_t *)k_porta_pdr_addr;  
00116 }
```



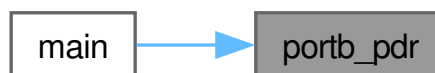
```
volatile uint8_t * porta_pdr (  
    void )    [inline], [static]
```

```
00118 {  
00119     return (volatile uint8_t *)k_porta_pdr_addr;  
00120 }
```



```
volatile uint8_t * portb_pdr (  
    void )    [inline], [static]
```

```
00122 {  
00123     return (volatile uint8_t *)k_portb_pdr_addr;  
00124 }
```



```
volatile uint8_t * portb_podr (  
    void )    [inline], [static]
```

```
00126 {  
00127     return (volatile uint8_t *)k_portb_podr_addr;  
00128 }
```



```

00001 /**
00002  * @file main.c
00003  * @brief RX72N multi-LED breathing cycle test: PA7, PB0, P71, P72, PB1, PB2.
00004  *
00005  * @details
00006  * Bare-metal test with no RTOS, no BSP, and no external clock configuration.
00007  * Six LEDs breathe sequentially using software PWM, then all flash together
00008  * to signal the end of one full cycle before repeating.
00009  *
00010  * Breathing sequence:
00011  * PA7 -> PB0 -> P71 -> P72 -> PB1 -> PB2 -> [flash all x3] -> repeat
00012  *
00013  * Runs on the default internal LOCO clock at 240 kHz (OFS1 = 0xFFFFFFFF).
00014  * Software PWM timing is calibrated for this clock.
00015  *
00016  * Port register addresses from RX72N Hardware Manual (r01uh0824ej0111):
00017  * PDR = 0x0008C000 + port_index (direction: 0=input, 1=output)
00018  * PODR = 0x0008C020 + port_index (output data)
00019  * Port indices: 7=0x07, A=0x0A, B=0x0B
00020  *
00021  * After reset, PMR registers default to 0x00 (all GPIO) so no MPC
00022  * unlock is required.
00023  *
00024  * Built with -O0 so volatile delay loops are not optimised out.
00025  *
00026  * SPDX-License-Identifier: MIT
00027  * @copyright Copyright (c) 2026 Locked Inc.
00028  */
00029
00030 #include <stdint.h>
00031
00032 /* -----
00033  * Port register addresses.
00034  *
00035  * uintptr_t is mandatory for hardware addresses: on the RX72N target it maps
00036  * to uint32_t, but on an x86 unit-test host it is uint64_t. Using
00037  * uint32_t directly would silently truncate on the 64-bit host.
00038  * ----- */
00039 typedef enum : uintptr_t {
00040     k_port7_pdr_addr = 0x0008C007U, /**< PORT7 direction register */
00041     k_port7_podr_addr = 0x0008C027U, /**< PORT7 output data register */
00042     k_porta_pdr_addr = 0x0008C00AU, /**< PORTA direction register */
00043     k_porta_podr_addr = 0x0008C02AU, /**< PORTA output data register */
00044     k_portb_pdr_addr = 0x0008C00BU, /**< PORTB direction register */
00045     k_portb_podr_addr = 0x0008C02BU, /**< PORTB output data register */
00046 } port_addr_t;
00047
00048 /* -----
00049  * Pin bit masks (one enum per port to avoid duplicate-value confusion).
00050  * ----- */
00051 typedef enum : uint8_t {
00052     k_pin_pa7 = (uint8_t)(1U « 7U), /**< PORTA pin 7 */
00053 } porta_pins_t;
00054
00055 typedef enum : uint8_t {
00056     k_pin_p71 = (uint8_t)(1U « 1U), /**< PORT7 pin 1 */
00057     k_pin_p72 = (uint8_t)(1U « 2U), /**< PORT7 pin 2 */
00058 } port7_pins_t;
00059
00060 typedef enum : uint8_t {
00061     k_pin_pb0 = (uint8_t)(1U « 0U), /**< PORTB pin 0 */
00062     k_pin_pb1 = (uint8_t)(1U « 1U), /**< PORTB pin 1 */
00063     k_pin_pb2 = (uint8_t)(1U « 2U), /**< PORTB pin 2 */
00064 } portb_pins_t;
00065
00066 /* -----
00067  * Breathing and flash timing.
00068  *
00069  * Calibration (LOCO = 240 kHz, -O0):
00070  * One delay() iteration = ~12 LOCO cycles = ~50 us.
00071  * PWM period = k_max_bright iterations = 50 * 50 us = 2.5 ms (400 Hz).
00072  * Each brightness step is held for k_reps_per_step PWM cycles:
00073  * 5 * 2.5 ms = 12.5 ms per step.
00074  * Full breathe (in + out) = 2 * 51 steps * 12.5 ms ~ 1.3 s per LED.
00075  * ----- */
00076 typedef enum : uint32_t {
00077     k_max_bright = 50U, /**< PWM duty-cycle upper bound (steps 0..50) */
00078     k_reps_per_step = 5U, /**< PWM cycles held at each brightness step */
00079     k_flash_count = 3U, /**< Completion flashes after one full cycle */
00080     k_flash_on_cnt = 8000U, /**< Flash ON busy-wait count */
00081     k_flash_off_cnt = 8000U, /**< Flash OFF busy-wait count */
00082 } breathe_params_t;
00083
00084 /* -----
00085  * LED count
00086  * ----- */

```

```

00087 typedef enum : uint8_t {
00088     k_num_leds = 6U, /**< LEDs in the breathing sequence */
00089 } led_count_t;
00090
00091 /* -----
00092  * LED descriptor
00093  * ----- */
00094 /**
00095  * @brief Binds a GPIO output register to a single pin's bit mask.
00096  */
00097 typedef struct {
00098     volatile uint8_t *podr; /**< Port output data register for this LED */
00099     uint8_t mask; /**< Bit mask selecting this LED's pin */
00100 } led_t;
00101
00102 /* -----
00103  * Inline port register accessors
00104  * ----- */
00105 static inline volatile uint8_t *port7_pdr(void)
00106 {
00107     return (volatile uint8_t *)k_port7_pdr_addr;
00108 }
00109 static inline volatile uint8_t *port7_podr(void)
00110 {
00111     return (volatile uint8_t *)k_port7_podr_addr;
00112 }
00113 static inline volatile uint8_t *porta_pdr(void)
00114 {
00115     return (volatile uint8_t *)k_porta_pdr_addr;
00116 }
00117 static inline volatile uint8_t *porta_podr(void)
00118 {
00119     return (volatile uint8_t *)k_porta_podr_addr;
00120 }
00121 static inline volatile uint8_t *portb_pdr(void)
00122 {
00123     return (volatile uint8_t *)k_portb_pdr_addr;
00124 }
00125 static inline volatile uint8_t *portb_podr(void)
00126 {
00127     return (volatile uint8_t *)k_portb_podr_addr;
00128 }
00129
00130 /* -----
00131  * Busy-wait delay
00132  * ----- */
00133 /**
00134  * @brief Spin for approximately @p count loop iterations.
00135  * @param[in] count Iteration count. Each iteration is ~50 us at LOCO 240 kHz.
00136  */
00137 static void delay(uint32_t count)
00138 {
00139     volatile uint32_t i;
00140     for (i = 0U; i < count; i++) {
00141         __asm__ __volatile__ ("nop");
00142     }
00143 }
00144
00145 /* -----
00146  * LED helpers
00147  * ----- */
00148 static inline void led_on(const led_t *led)
00149 {
00150     *led->podr |= led->mask;
00151 }
00152
00153 static inline void led_off(const led_t *led)
00154 {
00155     *led->podr &= (uint8_t)~led->mask;
00156 }
00157
00158 /* -----
00159  * Breathing effect
00160  * ----- */
00161 /**
00162  * @brief Run one PWM ramp pass on @p led.
00163  *
00164  * @details
00165  * Steps @p on_time from 0 to @c k_max_bright (or vice versa depending on
00166  * @p invert), holding each level for @c k_reps_per_step PWM cycles.
00167  *
00168  * @param[in] led LED to animate.
00169  * @param[in] invert When 0, ramp 0->max (fade in). When 1, ramp max->0 (fade out).
00170  */
00171 static void breathe_ramp(const led_t *led, uint32_t invert)
00172 {
00173     uint32_t b;

```

```

00174     uint32_t rep;
00175     uint32_t on_time;
00176     uint32_t off_time;
00177
00178     for (b = 0U; b <= k_max_bright; b++) {
00179         on_time = (invert == 0U) ? b : (k_max_bright - b);
00180         off_time = k_max_bright - on_time;
00181         for (rep = 0U; rep < k_reps_per_step; rep++) {
00182             if (on_time > 0U) {
00183                 led_on(led);
00184                 delay(on_time);
00185             }
00186             led_off(led);
00187             if (off_time > 0U) {
00188                 delay(off_time);
00189             }
00190         }
00191     }
00192 }
00193
00194 /**
00195  * @brief Run one full breathe cycle (fade in then fade out) on @p led.
00196  * @param[in] led LED to animate.
00197  */
00198 static void breathe(const led_t *led)
00199 {
00200     breathe_ramp(led, 0U); /* in */
00201     breathe_ramp(led, 1U); /* out */
00202     led_off(led);
00203 }
00204
00205 /* -----
00206  * Completion flash
00207  * ----- */
00208 /**
00209  * @brief Flash all LEDs @c k_flash_count times to signal cycle completion.
00210  * @param[in] leds LED descriptor array.
00211  * @param[in] count Number of entries in @p leds.
00212  */
00213 static void flash_all(const led_t *leds, uint8_t count)
00214 {
00215     uint32_t f;
00216     uint8_t i;
00217
00218     for (f = 0U; f < k_flash_count; f++) {
00219         for (i = 0U; i < count; i++) {
00220             led_on(&leds[i]);
00221         }
00222         delay(k_flash_on_cnt);
00223
00224         for (i = 0U; i < count; i++) {
00225             led_off(&leds[i]);
00226         }
00227         delay(k_flash_off_cnt);
00228     }
00229 }
00230
00231 /* -----
00232  * Entry point
00233  * ----- */
00234 /**
00235  * @brief Configure GPIO outputs and run the breathing cycle forever.
00236  * @return Never returns.
00237  */
00238 int main(void)
00239 {
00240     /* Set LED pins as outputs (PDR bit = 1). */
00241     *porta_pdr() |= (uint8_t)k_pin_pa7;
00242     *port7_pdr() |= (uint8_t)(k_pin_p71 | k_pin_p72);
00243     *portb_pdr() |= (uint8_t)(k_pin_pb0 | k_pin_pb1 | k_pin_pb2);
00244
00245     /* LED table -- order defines the breathing sequence. */
00246     const led_t leds[k_num_leds] = {
00247         { porta_podr(), (uint8_t)k_pin_pa7 },
00248         { portb_podr(), (uint8_t)k_pin_pb0 },
00249         { port7_podr(), (uint8_t)k_pin_p71 },
00250         { port7_podr(), (uint8_t)k_pin_p72 },
00251         { portb_podr(), (uint8_t)k_pin_pb1 },
00252         { portb_podr(), (uint8_t)k_pin_pb2 },
00253     };
00254
00255     for (;;) {
00256         uint8_t i;
00257         for (i = 0U; i < k_num_leds; i++) {
00258             breathe(&leds[i]);
00259         }
00260         flash_all(leds, k_num_leds);

```

00261 }
00262 }
